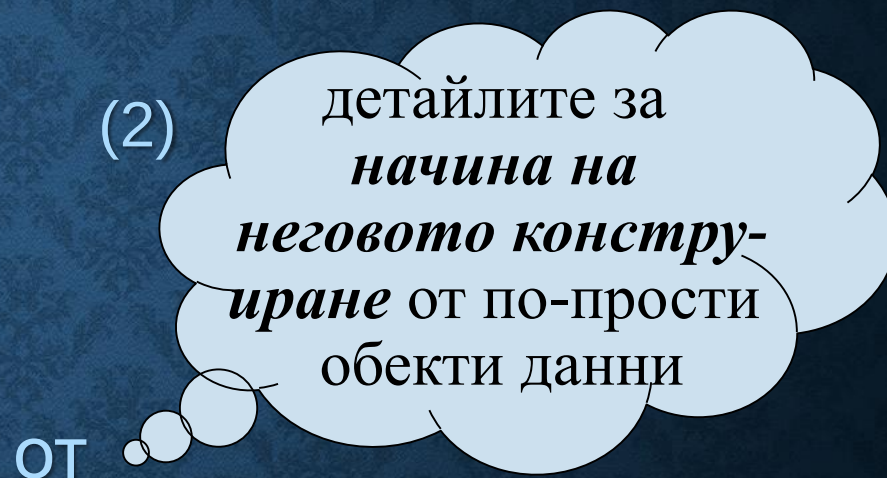
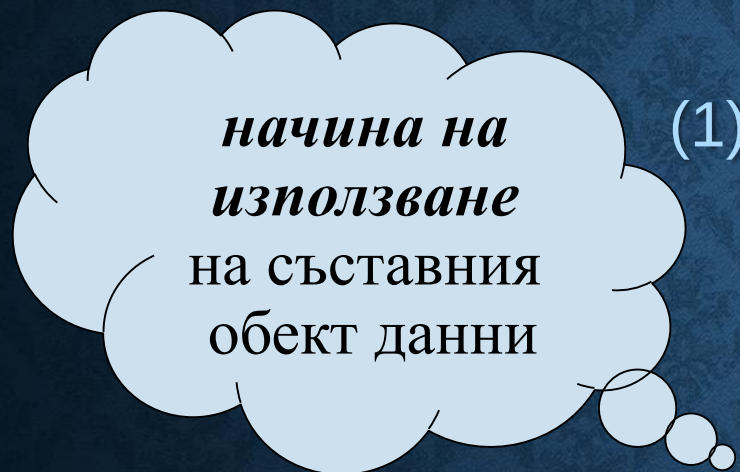


ТЕМА 6.

СЪСТАВНИ ТИПОВЕ ДАННИ

СТРУКТУРИ ОТ ДАННИ (СД) И АБСТРАКТНИ ТИПОВЕ ДАННИ (АТД)

Когато става дума за **съставни данни**, абстракцията на данните се използва като методология, която ни позволява да отделим (изолираме):



от

- (1) определя понятието **абстрактен тип данни** (АТД, *abstract data type*) – съвкупността от стойности на данните (областта) плюс спецификациите на операциите, използвани за създаване и обработка на данни
- (2) определя понятието **структура от данни** (СД, *data structure*) – съвкупността от елементи на данните, чиято организация се определя от операциите за достъп, които се използват за съхранение и извличане на отделните елементи на данните, т.е. СД имат две основни свойства:
 - могат да бъдат “разложени” на техните съставни части
 - подреждането на елементите е свойство на структурата, което се определя от това – какъв е достъпът до всеки елемент

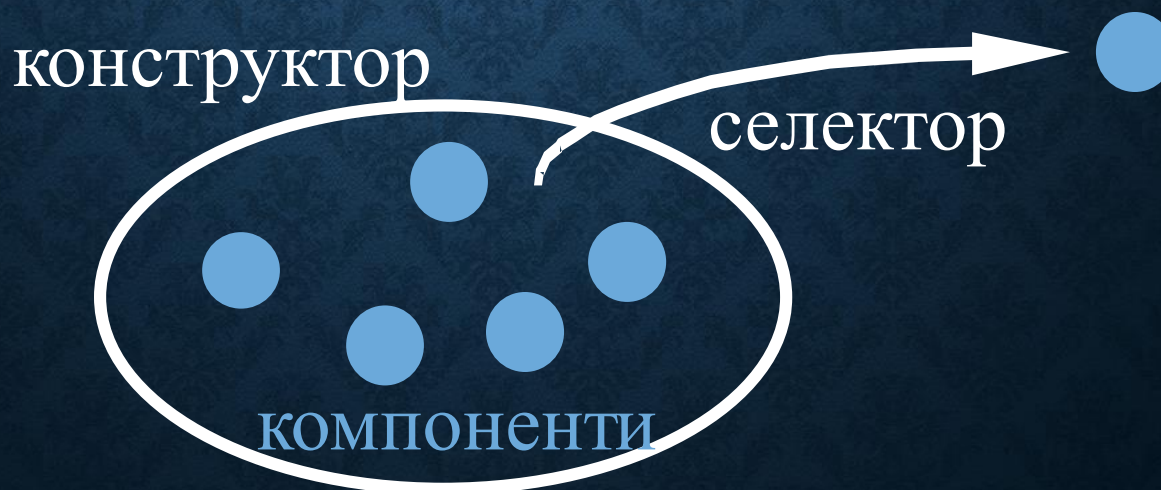
СЪСТАВНИ ТИПОВЕ ДАННИ: ОСНОВНИ ПОНЯТИЯ

- Съставните (комбинирани, сложните, *composite*) типове са такива, чийто стойности се състоят от **повече от една стойност**
- Съставлящите стойности се наричат **компоненти** (*components*) или **елементи** (*elements*)
- Компонентите принадлежат на други предварително определени типове. От тях по определени **правила за структуриране** (метод на структуриране) се конструират стойностите на сложните типове
- Доколкото типа на компонентите може също да бъде съставен, то може да се построи **йерархия от структури**, но крайните компоненти на всеки сложен тип са атомарни, т.е. са от прост тип и над тях могат да се извършват операциите, определени за съответния прост тип.

СЪСТАВНИ ДАННИ: ОПЕРАЦИИ

За всеки АТД могат да са определени различни операции за обработка на стойностите му, но задължително за всеки метод за структуриране съответства **уникална двойка операции**, изобразяващи типа на компонентите в съставния тип и обратно:

- **конструктор** (*construction*) – **определя как** съставната стойност се строи от стойностите на компонентите
- **селектор** (достъп до елементите, *selection, access*) – **определя как** стойностите на компонентите се извличат от съставната



СЪСТАВНИ АБСТРАКТНИ ТИПОВЕ ДАННИ: БАЗОВИ ОПЕРАЦИИ

Върху съставните типове най-често се прилагат следните основни операции:

- *добавяне* на компонент (*add*)
- *премахване* на компонент (*remove*)
- *търсене, извличане(в интервал)/копиране и сортиране* на компонент/и (*find, retrieve subset/copy, sort*)

Други операции разширяващи функционалността

- *проверка* дали има въобще компоненти (*is empty*)
- *премахване на всички компоненти* (*empty*) и др.



Тези операции са важни по отношение на реализацията на произволен алгоритъм, поради което са имплементирани в различните езици като стандартни библиотеки напр. C++ STL

СЪСТАВНИ АБСТРАКТНИ ТИПОВЕ ДАННИ: КЛАСИФИКАЦИЯ

Сложните типове най-често се класифицират според това дали:

- компонентите им са **подредени** – подредени и не подредени
- компонентите са стойности **от един и същи тип** – с еднотипни компоненти или и с разнотипни компоненти (комбинирани)
- **броят на компонентите** им е предварително фиксиран или не – статични и динамични
- според **механизмите за достъп** до компонентите им – с последователен (т.нар. последователности) и с директен достъп



Тези свойства са важни по отношение на тяхното използване при моделиране на данните

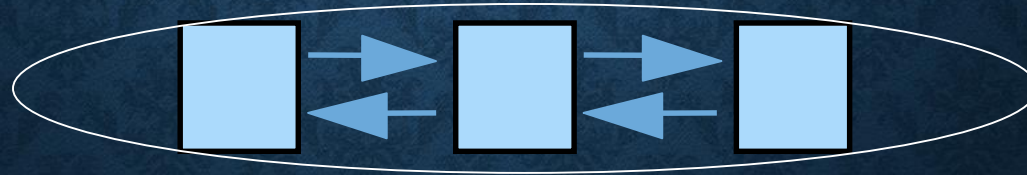
АТД СПОРЕД ПОДРЕДЕНОСТТА НА КОМПОНЕНТИТЕ

АТД е **подреден** (структуриран) ако за всяка компонента е известно коя е предходната (или предходните) и коя е следващата (или следващите).

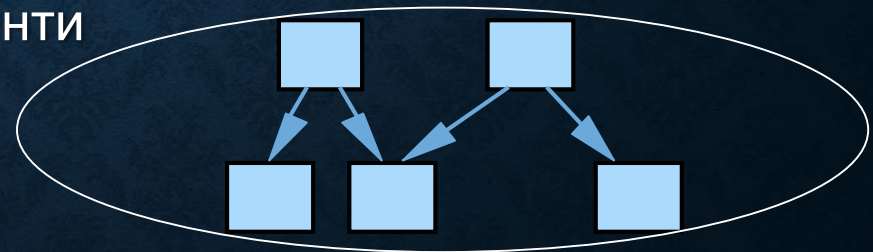
Ако реда на компонентите няма значение, т.е. елементите не са организирани по отношение един към друг, то съставният тип е **неподреден** (неструктуриран)

Подредените АТД биват:

- **линейни АТД** – всяка компонента има най-много един предходен и/или един следващ елемент (list, stack и queue)



- **нелинейни АТД** – всяка компонента може да има произволен брой предходни и/или следващи елементи (tree)



АТД СПОРЕД ТИПА НА КОМПОНЕНТИТЕ

- **АТД с еднотипни компоненти:** всички елементи на съставната стойност са от един и същи тип (array)

Типът на компонентите се нарича *базов тип* на съответния сложен тип

- **АТД с разнотипни компоненти:** компонентите на съставната стойност могат да принадлежат на различни предварително дефинирани типове (structure)

АТД СПОРЕД НАЧИНА НА ДОСТЪП ДО ЕЛЕМЕНТИТЕ

Класификацията “според начина на достъп до елементите” се отнася само за подредените АТД:

- АТД с **последователен достъп**: компонентите на съставната стойност могат да се обхождат единствено от началото към края (осигурено е средство, така че от всеки компонент може да се премине към следващ). Това означава, че за да извлечем някой елемент (да осъществим достъп до него) ще трябва да обходим всички елементи, намиращи се преди него

Пример: Представете си редица от стаи, като от всяка стая може да се преминава в следващата (преходни) и във всяка стая има един студент. За да намерим определен студент, който е в петата стая...

- АТД с **директен (пряк) достъп**: всеки компонент на съставната стойност е еднакво достъпен във всеки момент

Пример: Представете си множество от стаи и коридор, от който има врата към всяка стая

АТД СПОРЕД БРОЯ НА КОМПОНЕНТИТЕ

- **Статични АТД:** броят на компонентите на съставната стойност е предварително фиксиран и не може да се променя по време на съществуване на структурата (array)
- **Динамични АТД:** броят на компонентите на съставната стойност може да се променя по време на съществуване на структурата (list, stack и queue)

Забележка: За динамичните АТД с последователен достъп е характерно, че елементи могат да се добавят или изключват от съставната стойност само в единия край на структурата, докато за тези с директен достъп това може да се прави на произволно място от структурата



Много от съставните АТД са дефинирани в програмните езици като **вградени** съставни **типове**

АТД МАСИВ

Всяка стойност от **тип масив** (*array*) представлява подредена крайна съвкупност от еднотипни компоненти, до които е осигурен пряк достъп. Масивът е статичен АТД

- **Характеристики:**

- **краен** означава, че елементите са краен брой
- **статичен** означава че размерът на масива трябва да се знае по време на компилирането, но това не означава че всички места в масива трябва да съдържат смислени данни
- **подреден** означава, че относителното място на всеки елемент в масива е предварително дефиниран
- **еднотипни компоненти** означава, че елементите са от един и същ тип данни
- **прекия достъп** до елемент, означава, че при достъп до елемент на масива не е необходимо да се обръщаме към предишните елементи

- **Приложение:** Масивите се използват най-често за моделиране на подредени по някакъв признак еднотипни данни, които се подлагат на еднотипна обработка

- **Пример:** За студентите от даден курс често се налага да знаем кой студент е с най-висок среден успех, кои са студентите с отличен успех или какъв е средният успех на целия курс. В този случай е подходящо да се използва масив, като наредбата на компонентите (оценки на студентите) се извършва, например по поредния номер на студента в курса

АТД МАСИВ: ОСНОВНИ ПОНЯТИЯ

- Типът на компонентите се нарича **базов тип** на масива
- Броят на признаците определящи наредбата на елементите се нарича *размерност на масива (dimension)*. В зависимост от броя на признаците за наредба масивите се наричат съответно **едномерен**, **двумерен** и т.н., изобщо **многомерен масив**

Пример: В примера за студентите наредбата на компонентите се извършва по един признак. Възможно е наредбата да се определя от два признака (напр. оценките на студентите да са подредени по пореден номер студент и по изучавани дисциплини)

- Относителното място на един елемент на масива по отношение на някоя размерност се нарича **индекс на елемента (index)**

Примери за приложение на едномерни масиви: Те са естествена структура за съхранение на списъците от еднакви елементи на данните, като са списъци с покупки, ценови листи, списък с телефонни номера, списък с досиетата на студентите или списък от букви (низ)

АТД МАСИВ: СХЕМАТИЧНО ПРЕДСТАВЯНЕ

- Едномерен масив
(вектор, таблица, *vector*, *table*)

п о р е д е н н о м е р	4,25	6,00	...	...
-------------------------	------	------	-----	-----

- Двумерен масив

дисциплина п о р е д е н н о м е р	Л А А Г	У П	А н г л и й с к и
	п о р е д е н н о м е р	п о р е д е н н о м е р	п о р е д е н н о м е р
1	3.00	6.00	6.00
2			
3			

- Тримерен масив

Г О Д И Н А

Д И С Ц И П Л И Н А

П О Р Е Д Е Н Н О М Е Р

АТД МАСИВ: ДОСТЪП И КОНСТРУИРАНЕ

- **Достъп:**

Мястото на елемента в масива е строго определено от набора на всичките му индекси по всички размерности на масива

Желаният елемент се посочва чрез неговия **индекс** (или индекси), което дава относителното му място в съвкупността

- **Конструиране:**

Конструирането на масивите, т.е. изграждането на съвкупността от стойностите на неговите елементи най-често се извършва поелементно, чрез **обхождане** (*traversing*) на масива. Това означава че ние последователно се обръщаме към елементите (чрез техните индекси) и записваме необходимите данни в тях.

МАСИВИТЕ В ЕЗИЦИТЕ ЗА ПРОГРАМИРАНЕ

В почти всички ЕП от високо ниво масивите са предварително дефинирани като стандартен тип, както и операциите за тяхното конструиране и достъп до всеки отделен елемент

- **Дефиниране (Python):**

```
myList=[1,2,3,4,5,6]
```

Достъп — винаги се използват индексите с различни нюанси:

- до отделен елемент :

```
myList[2]=100
```

- до сегмент:

```
myList[2:5]
```

- **Конструиране:**

- последователно поелементно конструиране (чрез присвояване):

```
myList=[]
```

```
for i in range(10):
```

```
myList[i]=1
```

- В съвременните езици от високо ниво Java и C# работата с масивите е аналогична, дори и синтаксиса се припокрива:

```
int[] anArray;
```

```
anArray = new int[10];
```

```
anArray[0] = 100;
```

МАСИВИТЕ В ЕЗИЦИТЕ ЗА ПРОГРАМИРАНЕ

В Pascal, пример за декларация на два масива съответно от цели числа и булеви стойности и инициализация съответно със стойности 0 и false

Var

```
i      : Integer;  
myIntArray : Array[1..20] of Integer;  
myBoolArray : Array[1..20] of Boolean;
```

Begin

For i := 1 to 20 **do**

Begin

```
myIntArray[i] := 0;  
myBoolArray[i] := false;
```

End;

End.



Внимание! В някои ЕП на индексите се съпоставят цели числа в интервал 0, ..., n-1 докато в други интервала е 1, ..., n

АТД МАСИВ В C++ – ФМ И ЗМ

Физически модел

Компонентите на един масив се записват в последователни клетки от паметта като първо се разгъва най-вътрешната размерност. Масивът се съхранява в толкова байта, колкото (изисква типа на компонента) * (броя елементи)

Знаков модел

В C++ масивът е статична структура, т.е. броят на компонентите му е фиксиран и се задава при декларирането на типа

- Синтаксис за деклариране на променлива-масив (т.нар. **built-in array**):

<тип> <име> [<константен израз₁>_{незад}] [<константен израз₂>] ...

, където:

- *<тип>* е име или декларация на тип и определя базовия тип на масива, т.е. типа на неговите компоненти. Той може да бъде всеки тип освен `void`, псевдоним или тип-функция
- *<име>* е името на променливата-масив
- броят на участващите елементи от вида *[<константен израз>]* определя размерността на масива
- всеки *<константен израз_i>* е от целочислен тип и има стойност $N_i \geq 0$ и определя броят на елементите на масива по съответната размерност (посл. с индекс $= N_i - 1$)
- *<константен израз₁>* може да се пропусне само ако е извършено инициализиране на масива или ако той е деклариран като параметър, или в др. специални случаи

АТД МАСИВ В C++ – ПРИМЕРИ

(1)

```
float x[10]; // деклариране на масив, съдържащ 10 елемента
```

(2)

```
const int MAXCHILDREN = 500; // ! препоръчва се  
double heights[MAXCHILDREN];
```

(3)

```
float matrix[10][15]; // двумерен масив matrix
```

(4)

```
struct {           // едномерен масив от елементи от тип структура  
    float x, y;  
} complex[100];
```

(5) декларацията може да е смесена — обикновени променливи и масиви

```
const int MAXPOINTS = 50;  
double xcoord[MAXPOINTS], xmax, ycoord[MAXPOINTS],  
       ymax, zcoord[MAXPOINTS], zmax;
```

АТД МАСИВ В C++ – ПРИМЕРИ

(7)

`int a[5][5][5];` // дефинира масив, съдържащ 5 реда, 5 колони и всяка колона има по 5 елемента

```
for( int i(0); i < 5; i++ )  
    for( int j(0); j < 5; j++ )  
        for( int k(0); k < 5; k++ )  
            a[i][j][k]; //достъп до елементите,  
чрез 3 индекса
```

АТД МАСИВ В C++: МНОЖЕСТВО ОТ СТОЙНОСТИ

- **Множество от стойности:**

- Всяка стойност от даден масив представлява съвкупност от стойности от базовия тип, на брой $= N_1 * N_2 * \dots$ произведението на броевете по всички размерности на масива
- На последователните елементи по всяка размерност i , чрез *<константен израз_i>* се съпоставя индекс в интервала $[0, N_i - 1]$

- **Примери:**

За променливите-масиви декларирани по-горе, множествата от стойности са както следва:

(1)

```
float x[10]; // масив, съдържащ 10 елемента от тип float
           //с индекси 0, 1, ..., 8, 9
```

(3)

```
float matrix[10][15]; //двумерен масив matrix със 150 елемента
                    //с индекси 0, 1, ..., 8, 9 и 0, 1, ..., 8, 14
```

(4)

```
struct {           // едномерен масив от структури със 100 елемента
    float x, y;
} complex[100];
```

АТД МАСИВ В С++: ДЕФИНИРАНЕ НА МАСИВ ПО ВРЕМЕ НА ИЗПЪЛНЕНИЕ

Операторите **new** и **delete** съответно заделят (allocate memory) и освобождават (free memory) памет

```
#include <iostream>

int main() {
    using namespace std;
    int size, i = 0;
    cout<<"Number of elements";
    cin>>size;
    int* myarr = new int[size];

    for (i = 0 ; i < size ; i++)
        myarr[i] = 10;

    for (i = 0 ; i < size ; i++)
        cout<< myarr[i]);
    delete [] myarr;
}
```

АТД МАСИВ В C++: ОПЕРАЦИЯ СЕЛЕКТОР

- **Селектор** – Директният достъп към компонентите на масив се осъществява чрез указване стойността на индекса или индексите им (при многомерните масиви), чрез постфиксната операция [...]:

<име> [<константен израз₁>] [<константен израз₂>] ...

, където:

- *<име>* е името на предварително декларирана променлива-масив
- броят на участващите елементи от вида [*<константен израз>*] е = размерността на масива според неговата декларация
- всеки *<константен израз_i>* е от целочислен тип и определя индекса на елемента, до който се осъществява достъп по съответната размерност

- **Примери:**

```
x[4] = 0.5;  
cin >> x[4];  
value = x[n];  
cout << x[4];  
cin >> matrix[1][10];  
matrix[0][0] = 2.14;
```

АТД МАСИВ В С++: ОПЕРАЦИИ КОНСТРУКТОР И ИНИЦИАЛИЗИРАНЕ

- **Конструиране** – конструирането на масива може да се направи по два начина:

1. Чрез обхождане (*traversing*) на елементите:

```
for(int i=0; i<10; i++){  
    x[i] = float(i); // обхождане  
}  
  
double t_matrix[4][4];  
...  
for(int i = 0; i < 4; i++) {  
    for(int j = 0; j < 4; j++)  
        t_matrix[i][j] = 0.0; // по редове  
}
```

• 2. При инициализирането на масива

```
float z[3] = {1.5, 6.8, 4.3};  
double mydata[4][5] = {  
    { 1.0, 0.54, 0.21, 0.11, 0.03 },  
    { 0.34, 1.04, 0.52, 0.16, 0.09 },  
    { 0.41, 0.02, 0.30, 0.49, 0.19 },  
    { 0.01, 0.68, 0.72, 0.66, 0.17 }  
};
```

АТД МАСИВ В C++: ИНИЦИАЛИЗИРАНЕ – ОСОБЕНОСТИ

- Масив може да се инициализира с празно множество

```
int heights [5] = { }; //масив с нулирани елементи
```

- При инициализирането на масива могат да се пропуснат вътрешните скоби {}:

```
double mydata[4][5] = {  
    1.0, 0.54, 0.21, 0.11, 0.03, 0.34, 1.04, 0.52, 0.16, 0.09,  
    0.41, 0.02, 0.30, 0.49, 0.19, 0.01, 0.68, 0.72, 0.66, 0.17  
};
```

- При инициализирането на масива, ако не се зададе размер на масива компилаторът ще запълни масива според размера на инициализиращия списък:

```
int dayPerMonth[]={31,28,31, 30, 31, 30, 31, 31, 30, 31,30,31}; //  
дефинира масив от 12 int елемента
```

- При инициализирането на многомерен масив може да не се зададе размерът само по първото измерение на масива:

```
double mydata[][5] = {  
    { 1.0, 0.54, 0.21, 0.11, 0.03 },  
    { 0.34, 1.04, 0.52, 0.16, 0.09 },  
    { 0.41, 0.02, 0.30, 0.49, 0.19 },  
    { 0.01, 0.68, 0.72, 0.66, 0.17}  
};
```

АТД МАСИВ В C++: ДРУГИ ОПЕРАЦИИ

- Други операции с масиви не са дефинирани
- Т.е. всички други обработки с масив се извършват поелементно с използване на операцията селектор
- Тъй като елементите принадлежат на някой от базовите типове, то всички операции дефинирани за този базов тип са приложими върху елементите на масива

АТД МАСИВ В C++: ОСОБЕНОСТИ

- Както в С, така и в C++ масивите всъщност са последователно разположени клетки в паметта и поради това могат да бъдат обработвани и чрез използване на **указатели и аритметика** с тях

```
int main()

{ int number[] = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31,
30, 31 };

  int *pNumbers = number;

  cout << "\n*pNumbers: " << *pNumbers;

  cout << "\n*(pNumbers+1): " << *(pNumbers+1);

  cout << "\n*(pNumbers+2): " << *(pNumbers+2);

  cout << "\n*(pNumbers+3): " << *(pNumbers+3);

  cout << "\n*(pNumbers+4): " << *(pNumbers+4);

}
```

АТД МАСИВ В C++: ОСОБЕНОСТИ

- При работа с масиви има опасност да се излезе извън неговия диапазон. Трябва да се има пред вид, че **компиляторът не извършва проверки** и избягването на тези грешки е оставено на грижата на програмиста

```
double distance[] = {44.14, 720.52, 96.08, 468.78, 6.28};
```

```
cout << "2nd = " << distance[1] << endl;
```

```
cout << "6th = " << distance[5] << endl; // компилатора не посочва  
грешка
```

- Могат да се правят т.нар. **“непълни” декларации на масиви** – без да се указва броят на елементите на масива по първата размерност. По-късно в рамките на същата област на видимост тази декларация се попълва:

```
int incomp[][3]; //непълна декларация на масив  
void main()  
{ ...  
    int incomp[4][3]; //тук вече е попълнена  
}
```

МАСИВИ В C++ - ARRAY CLASS

- Шаблонен клас – контейнер с фиксиран брой елементи
- За използване трябва да се използва библиотеката <array> (от версия C++11)
- Синтаксис за деклариране

```
array<int, 3> a = {1,2,3};
```

- Налични са множество член функции, като size(), sort(), begin(), fill() и др.

- Пример

```
#include <iostream>
#include <array>
using namespace std;
```

```
int main()
{
    array<int,3> myarray {10,20,30};

    for (int i=0; i<myarray.size(); ++i)
        ++myarray[i]; // достъп, аналогичен на вградения тип, с [ ]

    for (int elem : myarray)
        cout << elem << '\n';
}
```

ПРИМЕР

```
const int NUM_QUIZZES = 10; //брой оценки
const int NUM_STUDENTS = 5; // брой студенти
int class_grades[NUM_STUDENTS][NUM_QUIZZES]; // масив за оценките
int student, quiz, quiz_sum;
double quiz_average;
cout << "Enter exactly: " << NUM_QUIZZES
<< " marks for each student." << endl;
for (student = 0; student < NUM_STUDENTS; ++student)
{
    cout << "Marks for Student " << student + 1 << ": ";
    for (quiz = 0; quiz < NUM_QUIZZES; ++quiz)
        cin >> class_grades[student][quiz];
}
// Изчислява и извежда средния успех на всеки студент
for (student = 0; student < NUM_STUDENTS; ++student)
{
    quiz_sum = 0;
    for (quiz = 0; quiz < NUM_QUIZZES; ++quiz)
        quiz_sum += class_grades[student][quiz];
    quiz_average = (double) quiz_sum / NUM_QUIZZES; // явно преобразуване на тип
    cout << endl;
    cout << "Student: " << setw(3) << student + 1
    << " Average: " << setw(5) << quiz_average;
}
```

**ДА СЕ НАПИШЕ ПРОГРАМА, КОЯТО ИЗЧИСЛЯВА СЕДМИЧНИТЕ ПРОДАЖБИ
И НАМИРА И ИЗВЕЖДА НАЙ-ВИСОКАТА СТОЙНОСТ ЗА СЕДМИЦАТА И В
КОЙ ДЕН Е ТЯ.**

```
#include <iostream>
#include <iomanip>
using namespace std;
void main()
{
    char day_names[7][10] = {"Monday", "Tuesday", "Wednesday",
    "Thursday", "Friday", "Saturday", "Sunday"};
    double sales[7]; //масив за съхраняване на стойностите на
    продажбите
    char salesperson[41]; // име на продавача
    double max_sales, total_sales;
    int day, max_day;
    cout << "Enter the name of the salesperson: ";
    cin.getline(salesperson, 41); //чете брой символи от
    клавиатурата
```

```

for (day = 0; day < 7; ++day)
{
    cout << "Enter the sales for "<< day_names[day] << ": ";
    cin >> sales[day];
}
total_sales = 0;
max_day = 0; //индекс на ден
max_sales = sales[0];
for (day = 0; day < 7; ++day)
{
    if (sales[day] > max_sales)
    {
        max_sales = sales[day];
        max_day = day;
    }
    total_sales += sales[day];
}
cout << "The total sales for " << salesperson
<< " is " << total_sales << "." << endl << endl;
cout << "The highest sales was " << max_sales << "." << endl
<< endl;
cout << "The highest sales occurred on "<<
day_names[max_day]<< "." << endl;
}

```